

6.2.3 UCS2/UTF-16

16-bit character data. Each 16-bit unit is transmitted **with the *b1* “first”, up to *b16* “last”**. (NOTE: at a lower protocol level: in 16-QAM 4 bits are sent in parallel, 64-QAM: 6 bits in parallel, 256-QAM: 8 bits in parallel, 1025-QAM: 10 bits in parallel.) **Though nominally big-endian, it is bit-wise little endian (so if each octet is ‘reversed’ individually (instead of reversing 16-bit units), the encoding is little-endian octet-wise...).**

- Bits per character: 16 (bits per code unit; a character in planes 1-17 are coded by 2 16-bit code units; **the byte order is big-endian if 16-bit units ‘reversed’, little-endian if 8-bit units ‘reversed’**)
- Pad character: CR (as 0x000D in 16 bits)
- Character table: ISO/IEC 10646 [10]; UTF-16, 1 to 2 hexadecets per character

UTF-16 is an extension of UCS2, plane 0, that allows for representing characters in planes 1-16.

When splitting a message text into submessages (using this encoding), there must be no cut inside of a 16-bit unit (if UCS2 is supported; since the max number of message bytes is even for each submessage both for SMS and CBS, this will not happen), nor between “high surrogate” and a “low surrogate” (the pair encodes for a character in plane 1 to 17; if UTF-16 is supported). (There should be no “surrogate” not in such a pairs; that would be an encoding error.)

Note that for CBS, a 2-letter language tag prefix for UCS2/UTF-16 encoded messages are encoded in the 7-bit 0x00 alphabet in the first two bytes; it should be lowercase.

Furthermore, if any of the 7-bit alphabets 0x10 and onwards can be used for an SMS/CBS message, use of that 7-bit alphabet is preferred, as that can more than half the length (in bytes) of the message, allowing for fewer submessages and allowing a longer (in characters) total message. (Note that alphabets 0x00-0x0F are deprecated for SMS/CBS messages.)

For messages originally encoded in a UCS encoding, the following mappings should be used, both upon sending (before encoding in a 7-bit encoding, if that is to be used), as well as upon receiving a UCS2 SMS/CBS message:

- U+0000-U+0008, U+000E-U+001F, U+007F-U+009A, U+009C-U+00A0: map to U+FFFD.
- U+0009: map to U+0020. U+000B, U+2028-U+2029: map to U+000A. U+061C: map to U+200F.
- U+2000-U+200B: map to U+FFFD.
- If in the immediate vicinity of an Arabic letter: U+200C: map to U+00B7 (use middle dot as non-joiner). U+200D: map to U+0640 (use kashida as joiner). Note that U+200D and U+200C have special handling for several Indic scripts, including when converting to 7-bit alphabets (see section A.4).
- U+202A-U+202E, U+205F-U+206F, U+D800-U+F8FF, U+FDD0-U+FDEF, U+FEFF, U+FFFC, U+xFFFE-U+xFFFF (where x is in $\{0, \dots, 10_{16}\}$): map to U+FFFD.

In case there are any bidi RTL characters in the message, a *simplified* Unicode bidi algorithm should be used before display of the message. Bidi AL is handled as bidi R, except Arabic punctuation which is handled as ON; all bidi EN and AN handled as bidi L; all bidi ET, ES, CS handled as bidi ON. Furthermore, the 'paragraph direction' is always LTR for SMS/CBS messages. Note also that there are no nestable bidi controls in SMS/CBS (or they are not interpreted in the simplified bidi algorithm, in case the mappings above have not been applied). Furthermore, Arabic is a 'cursive' script and Arabic characters thus should be cursive shaped (initial, medial, final, isolated forms), and both Arabic and Hebrew may have (certain) combining characters applied. Bidi mirroring (when displaying RTL text sections) applies, but only those that can be done via using the data in BidiMirroring.txt (Unicode).

Brahmic (Indic) scripts (if supported) also should be handled specially for rendering; the details of which are beyond the scope of this standard. Further, while most 'emoji' characters are isolated small coloured drawings, many emoji are made from 'emoji sequences', multiple emoji characters creating one emoji to display.

For the Latin script alphabet (“European”, identifier 0x10) most text for “European” languages (and Vietnamese) can be handled by “precomposed” letters in ISO/IEC 10646 (UCS). However, to be able to fit the Latin characters for these languages into one 7-bit alphabet (7-bit alphabet identifier 0x10), we rely combining characters, in particular for East European languages and Vietnamese. Similarly for polytonic Greek (modern Greek orthography is monotonic Greek, and that is covered without the need for combining characters in the 7-bit alphabet for Greek, identifier 0x11). The easiest way to handle that is to upon reception of a message in Latin (0x10) or Greek (0x11) 7-bit alphabet, is to first convert to ISO/IEC 10646 and then apply the Unicode Normal Form C (NFC). Upon sending, first apply NFC, then decompose letters that need to be represented in decomposed form in the Latin or Greek 7-bit alphabet.